Assignment 6: Implementation of Data Integrity Constraints

Hospital Management System - PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL and DEFAULT with verification

1. Objective

This assignment applies integrity constraints to the Hospital Management System, explains the purpose of each, and then proves with SQL that the database **rejects** invalid data and **preserves** relationships. The complete DDL is in Assignment 5; this report concentrates on the constraints.

2. Constraints Implemented and Their Purpose

Constraint	Where applied	Purpose
PRIMARY KEY	Every table (composite on PRESCRIPTION_MEDICINE)	Uniquely identifies each row and prevents duplicate or NULL identifiers (entity integrity). The composite key stops the same medicine being listed twice on one prescription.
FOREIGN KEY	patient_id, doctor_id, department_id, admission_id, prescription_id, medicine_id columns	Guarantees every reference points to an existing parent row (referential integrity), so there are no appointments for unknown patients, no doctors in non-existent departments, etc.
ON DELETE RESTRICT / CASCADE	All FKs; CASCADE only on PRESCRIPTION_MEDICINE.prescription_id	RESTRICT blocks deleting a patient/doctor/department that has history. CASCADE removes line items automatically when their prescription is deleted.
NOT NULL	Mandatory attributes (names, dates, FKs, amounts)	Ensures essential facts are always recorded. Only discharge_date, address, treatment and BILL.admission_id may be NULL.
UNIQUE	DEPARTMENTS.department_name; DOCTOR.ph_no, DOCTOR.email; PATIENTS(name, DOB, ph_no); MEDICINES(medicine_name, manufacturer)	Prevents duplicate departments, doctor contacts, duplicate patient registration and duplicate medicine entries. Patient phone alone is not unique because family members share numbers.
CHECK	Gender, appointment_status, payment_status, DOB, ph_no length, email format, discharge_date >= admission_date, total_amount >= 0, price > 0, stock_quantity >= 0, quantity > 0	Enforces valid values and business rules (domain integrity): no negative prices or stock, no discharge before admission, only permitted statuses.
DEFAULT	appointment_status = 'Scheduled'; payment_status = 'Pending'; bill_date, record_date, prescription_date = current date; stock_quantity = 0; quantity = 1	Supplies sensible values automatically when a column is omitted, reducing data-entry errors.

3. Verification Tests

Each test is executed inside a transaction that is rolled back afterwards, so the sample data is unchanged. Outputs are the actual messages returned by the database engine (SQLite wording; MySQL reports equivalent errors such as 1062 duplicate entry, 1452 foreign key failure, 3819 check constraint violated). **REJECTED** means the engine refused the statement, which is the expected result for invalid data.

Test 1: Primary key: duplicate patient_id

```
INSERT INTO PATIENTS (patient_id, name, DOB, Gender, ph_no, address)
VALUES (1001, 'Test Person', '1990-01-01', 'Male', '9000000001', 'Bengaluru');
```

Result: REJECTED - `UNIQUE constraint failed: PATIENTS.patient_id`

Test 2: Foreign key: appointment for a patient that does not exist

```
INSERT INTO APPOINTMENT (appointment_id, appointment_date, appointment_status, patient_id, doctor_id)
VALUES (5021, '2026-11-05', 'Scheduled', 9999, 101);
```

Result: REJECTED - `FOREIGN KEY constraint failed`

Test 3: Foreign key: doctor in a non-existent department

```
INSERT INTO DOCTOR (doctor_id, doctor_name, specialization, ph_no, email, department_id)
VALUES (111, 'Dr. Test', 'Dentist', '8041020099', 'test.doc@cityhospital.in', 99);
```

Result: REJECTED - `FOREIGN KEY constraint failed`

Test 4: Unique: duplicate doctor e-mail

```
INSERT INTO DOCTOR (doctor_id, doctor_name, specialization, ph_no, email, department_id)
VALUES (111, 'Dr. Test', 'Dentist', '8041020099', 'anil.kumar@cityhospital.in', 1);
```

Result: REJECTED - `UNIQUE constraint failed: DOCTOR.email`

Test 5: Unique: patient registered twice (same name, DOB, phone)

```
INSERT INTO PATIENTS (patient_id, name, DOB, Gender, ph_no, address)
VALUES (1016, 'Ravi Shankar', '1985-03-12', 'Male', '9845010001', 'Jayanagar, Bengaluru');
```

Result: REJECTED - `UNIQUE constraint failed: PATIENTS.name, PATIENTS.DOB, PATIENTS.ph_no`

Test 6: Unique: duplicate department name

```
INSERT INTO DEPARTMENTS (department_id, department_name, location)
VALUES (7, 'Cardiology', 'Block D');
```

Result: REJECTED - `UNIQUE constraint failed: DEPARTMENTS.department_name`

Test 7: Not null: patient without a name

```
INSERT INTO PATIENTS (patient_id, name, DOB, Gender, ph_no)
VALUES (1016, NULL, '1990-01-01', 'Male', '9000000002');
```

Result: REJECTED - `NOT NULL constraint failed: PATIENTS.name`

Test 8: Check: invalid gender value

```
INSERT INTO PATIENTS (patient_id, name, DOB, Gender, ph_no)
VALUES (1016, 'Test Person', '1990-01-01', 'Unknown', '9000000003');
```

Result: REJECTED - `CHECK constraint failed: Gender IN ('Male','Female','Other')`

Test 9: Check: discharge date earlier than admission date

```
INSERT INTO ADMISSIONS (admission_id, admission_date, discharge_date, room_number, patient_id, doctor_id)
VALUES (7009, '2026-10-10', '2026-10-05', 'A-101', 1001, 101);
```

Result: REJECTED - `CHECK constraint failed: chk_discharge`

Test 10: Check: invalid payment status

```
INSERT INTO BILL (bill_id, total_amount, payment_status, bill_date, patient_id)
VALUES (9010, 500.00, 'Unpaid', '2026-10-20', 1001);
```

Result: REJECTED - `CHECK constraint failed: payment_status IN ('Pending','Partial','Paid')`

Test 11: Check: negative medicine price

```
INSERT INTO MEDICINES (medicine_id, medicine_name, manufacturer, price, stock_quantity)
VALUES (212, 'Test Tablet', 'TestCo', -5.00, 10);
```

Result: REJECTED - `CHECK constraint failed: price > 0`

Test 12: Check: prescription quantity of zero

```
INSERT INTO PRESCRIPTION_MEDICINE (prescription_id, medicine_id, quantity)
VALUES (4003, 202, 0);
```

Result: REJECTED - `CHECK constraint failed: quantity > 0`

Test 13: Primary key (composite): same medicine twice on one prescription

```
INSERT INTO PRESCRIPTION_MEDICINE (prescription_id, medicine_id, quantity)
VALUES (4001, 204, 10);
```

Result: REJECTED - `UNIQUE constraint failed: PRESCRIPTION_MEDICINE.prescription_id, PRESCRIPTION_MEDICINE.medicine_id`

Test 14: Restrict: delete a doctor who has appointments

```
DELETE FROM DOCTOR WHERE doctor_id = 101;
```

Result: REJECTED - `FOREIGN KEY constraint failed`

Test 15: Restrict: delete a department that has doctors

```
DELETE FROM DEPARTMENTS WHERE department_id = 1;
```

Result: REJECTED - `FOREIGN KEY constraint failed`

Test 16: Restrict: delete an admission that has a bill

```
DELETE FROM ADMISSIONS WHERE admission_id = 7001;
```

Result: REJECTED - `FOREIGN KEY constraint failed`

Test 17: DEFAULT constraints

Columns are omitted on purpose; the database fills them with the DEFAULT values (transaction rolled back afterwards).

```
INSERT INTO APPOINTMENT (appointment_id, appointment_date, patient_id, doctor_id)
VALUES (5021, '2026-11-05', 1002, 108);

INSERT INTO BILL (bill_id, total_amount, patient_id) VALUES (9010, 2500.00, 1001);

INSERT INTO MEDICINES (medicine_id, medicine_name, manufacturer, price)
VALUES (212, 'Cetirizine 10mg', 'Cipla', 2.00);

SELECT appointment_id, appointment_status FROM APPOINTMENT WHERE appointment_id = 5021;
```

appointment_id	appointment_status
5021	Scheduled

1 row(s) returned

```
SELECT bill_id, payment_status, bill_date FROM BILL WHERE bill_id = 9010;
```

bill_id	payment_status	bill_date
9010	Pending	2026-10-23

1 row(s) returned

```
SELECT medicine_id, stock_quantity FROM MEDICINES WHERE medicine_id = 212;
```

medicine_id	stock_quantity
212	0

1 row(s) returned

Test 18: ON DELETE CASCADE on PRESCRIPTION_MEDICINE

```
SELECT COUNT(*) AS line_items FROM PRESCRIPTION_MEDICINE WHERE prescription_id = 4005;
```

line_items

3

1 row(s) returned

```
DELETE FROM PRESCRIPTION WHERE prescription_id = 4005;

SELECT COUNT(*) AS line_items FROM PRESCRIPTION_MEDICINE WHERE prescription_id = 4005;   -- run again after the delete
```

line_items
0

1 row(s) returned

Result: deleting prescription 4005 automatically removed its 3 line items, so no orphan line items remain. The delete was rolled back.

Test 19: Referential integrity check across the whole database

```
SELECT 'APPOINTMENT -> PATIENTS' AS relationship, COUNT(*) AS orphan_rows
FROM APPOINTMENT a LEFT JOIN PATIENTS p ON a.patient_id = p.patient_id WHERE p.patient_id IS NULL
UNION ALL
SELECT 'APPOINTMENT -> DOCTOR', COUNT(*)
FROM APPOINTMENT a LEFT JOIN DOCTOR d ON a.doctor_id = d.doctor_id WHERE d.doctor_id IS NULL
UNION ALL
SELECT 'DOCTOR -> DEPARTMENTS', COUNT(*)
FROM DOCTOR d LEFT JOIN DEPARTMENTS x ON d.department_id = x.department_id WHERE x.department_id IS NULL
UNION ALL
SELECT 'ADMISSIONS -> PATIENTS', COUNT(*)
FROM ADMISSIONS a LEFT JOIN PATIENTS p ON a.patient_id = p.patient_id WHERE p.patient_id IS NULL
UNION ALL
SELECT 'BILL -> ADMISSIONS (non-NULL)', COUNT(*)
FROM BILL b LEFT JOIN ADMISSIONS a ON b.admission_id = a.admission_id
WHERE b.admission_id IS NOT NULL AND a.admission_id IS NULL
UNION ALL
SELECT 'PRESCRIPTION_MEDICINE -> MEDICINES', COUNT(*)
FROM PRESCRIPTION_MEDICINE pm LEFT JOIN MEDICINES m ON pm.medicine_id = m.medicine_id
WHERE m.medicine_id IS NULL;
```

relationship	orphan_rows
APPOINTMENT -> PATIENTS	0
APPOINTMENT -> DOCTOR	0
DOCTOR -> DEPARTMENTS	0
ADMISSIONS -> PATIENTS	0
BILL -> ADMISSIONS (non-NULL)	0
PRESCRIPTION_MEDICINE -> MEDICINES	0

6 row(s) returned

```
PRAGMA foreign_key_check;   -- returns no rows when every foreign key is satisfied
```

table	rowid	parent	fkid
(no rows)			

0 row(s) returned

Result: every relationship reports 0 orphan rows and the engine's own foreign-key check returns nothing, confirming referential integrity holds.

4. Summary of Results

#	Test	Constraint	Result
1	Primary key: duplicate patient_id	PRIMARY KEY	**REJECTED**
2	Foreign key: appointment for a patient that does not exist	FOREIGN KEY	**REJECTED**
3	Foreign key: doctor in a non-existent department	FOREIGN KEY	**REJECTED**
4	Unique: duplicate doctor e-mail	UNIQUE	**REJECTED**
5	Unique: patient registered twice (same name, DOB, phone)	UNIQUE (composite)	**REJECTED**
6	Unique: duplicate department name	UNIQUE	**REJECTED**
7	Not null: patient without a name	NOT NULL	**REJECTED**
8	Check: invalid gender value	CHECK	**REJECTED**

#	Test	Constraint	Result
9	Check: discharge date earlier than admission date	CHECK	**REJECTED**
10	Check: invalid payment status	CHECK	**REJECTED**
11	Check: negative medicine price	CHECK	**REJECTED**
12	Check: prescription quantity of zero	CHECK	**REJECTED**
13	Primary key (composite): same medicine twice on one prescription	PRIMARY KEY	**REJECTED**
14	Restrict: delete a doctor who has appointments	ON DELETE RESTRICT	**REJECTED**
15	Restrict: delete a department that has doctors	ON DELETE RESTRICT	**REJECTED**
16	Restrict: delete an admission that has a bill	ON DELETE RESTRICT	**REJECTED**
17	Omitted columns receive default values	DEFAULT	**WORKS** - 'Scheduled', 'Pending', today's date, stock 0
18	Delete a prescription - its line items go with it	ON DELETE CASCADE	**WORKS**
19	Orphan-row scan and foreign_key_check	FOREIGN KEY	**0 violations**

5. Conclusion

Primary keys identify rows, foreign keys keep the tables consistent with each other, UNIQUE avoids duplicates, CHECK blocks invalid values, NOT NULL protects mandatory data, and DEFAULT fills in routine values. All invalid operations tested were rejected by the database itself, so data quality does not depend on application code alone.